A Multi-Model Benchmarking Framework for Real-World Credit Card Fraud Detection

1st Mukesh Kumar Bharti
School of Computer Science Engineering and Technology
Bennett University
Greater Noida, India
ORCID: 0009-0000-4073-1463

2nd Vaibhav Chandola
School of Computer Science Engineering and Technology
Bennett University
Greater Noida, India
ORCID: 0009-0004-7611-2735

3rd Krishan Kant Kashyap
School of Computer Science Engineering and Technology
Bennett University
Greater Noida, India
ORCID: 0009-0008-5305-8499

4th Vaibhav Sharma
School of Computer Science Engineering and Technology
Bennett University
Greater Noida, India
ORCID: 0009-0001-6132-8128

***Abstract*—This paper addresses the growing complexity of credit card fraud that comes from transactions by testing machine learning models to see how well the models can automatically find fraud and cut down false alarms. The study examined nine algorithms. The algorithms include Logistic Regression, Decision Tree, Random Forest, Multi-Layer Perceptron (MLP) Neural Networks, K-Nearest Neighbors (KNN), CatBoost, Naive Bayes, LightGBM, XGBoost. The algorithms were applied to a public transaction dataset. Models were evaluated with cross-validation by the use of accuracy, precision, recall, F1-score, Receiver Operating Characteristic - Area Under the Curve (ROC-AUC), and Precision-Recall - Area Under the Curve (PR-AUC) as metrics. The results show that the ensemble method, - XGBoost, shows the best detection capability and the most balanced classification performance. The findings point out that the class imbalance must be fixed. The findings also indicate that ensemble learning works well for real fraud detection.**

***Index Terms*—Credit Card Fraud Detection, Machine Learning, Ensemble Methods, XGBoost, F1-Score, ROC-AUC, PR-AUC.**

I. Introduction

It has been analyzed that credit card fraud is becoming more clever as digital payment methods grow. The growth of payments poses challenges for financial institutions. Financial institutions must spot suspicious transactions hidden in amounts of normal activity. Detection mechanisms need to catch fraud cases. Detection mechanisms must also keep alarms low so that analysts are not overloaded. Detection mechanisms must also deal with the difference between the number of fraud transactions and real transactions. Detection mechanisms must also change quickly to keep up with behaviors. [1], [2].

To address theses limitations, the researchers explore strategies by putting the data quality first. Researchers use data augmentation and resampling methods, such, as Synthetic Minority Over-Sampling Technique (SMOTE) and synthetic data creation to reduce class imbalance. Although the techniques improve the identification of the minority class, the techniques do not always improve the model generalizability. The impact of these techniques varies across datasets and model architectures [3], [4]. Traditional machine learning models, such as, logistic regression, random forests and boosting methods are clear and easy to use but often cannot capture feature interactions or spot behavioral anomalies that indicate fraud [5], [6].

Deep learning uses representation learning to improve detection. Researchers have used autoencoders, Convolutional Neural Network (CNN), Recurrent Neural Network (RNN) hybrids, and ensemble models to improve performance. The raised performance shows up especially on large-scale datasets, such as those listed in [7]–[9]. Recent methods that add uncertainty estimation lower the prediction overconfidence. Lower prediction overconfidence creates decision boundaries, as shown in [10]. Deep learning still shapes fraud detection frameworks. The results are not always the same and; change with the dataset and context. Hybrid architectures that use a Generative Adversarial Network (GAN) to create synthetic samples add flexibility. However, hybrid architectures also cause problems with data authenticity and the use of the system [11]–[13].

Despite these contributions, a critical gap still exists. The critical gap is the lack of a testing framework. The test framework must cover models, deep learning models, hybrid models, and augmented models. The test framework must use the evaluation rules. Existing research usually looks at model types. Existing research often uses metrics that cannot be compared with each other. Existing research also relies on datasets that do not reflect transaction flows [14], [15].

This work is designed to address this gap. A multi-model evaluation framework is introduced. The framework benchmarks many fraud detection approaches; in a controlled test environment. The framework lines up preprocessing steps, resampling methods, evaluation metrics, and validation protocols. The framework then provides a view of the model performance in settings that match real-world constraints.

TABLE I: List of Abbreviations

Abbreviation	Full Form
MLP	Multi-Layer Perceptron
KNN	K-Nearest Neighbors
ROC-AUC	Receiver Operating Characteristic – Area Under the Curve
PR-AUC	Precision-Recall – Area Under the Curve
PCA	Principal Component Analysis
SMOTE	Synthetic Minority Over-sampling Technique
XGBoost	Extreme Gradient Boosting
LightGBM	Light Gradient Boosting Machine
CatBoost	Categorical Boosting

The primary objectives of this research include the following:

- Designing a comprehensive benchmarking framework that integrates classical machine learning, deep learning, hybrid models, and augmentation-based techniques.
- Evaluating the performance of models through standardized metrics such as ROC-AUC, PR-AUC, recall, precision, and F1 score.
- Investigating the balance between interpretability, robustness, and generalizability across various modeling approaches.

II. Background

A. *Evolution of Credit Card Fraud Detection*

Credit card fraud detection has progressed from rule-based methods to machine learning models that are capable of handling complex transaction patterns. Early models, such as logistic regression and decision trees were interpretable but struggled with non-linear behavior and extreme class imbalances. Deep learning approaches improve accuracy through hierarchical feature learning but require large labeled datasets, high computation, and lack transparency.

Class imbalance has been mitigated using oversampling techniques, such as SMOTE, which improve minority detection but generalize inconsistently. Unsupervised methods using Principal Component Analysis (PCA) or autoencoder reconstruction errors are helpful when labels are scarce, although high false-positive rates limit their standalone use. These challenges underscore the need for a unified benchmarking framework to evaluate different approaches under realistic, imbalanced conditions.

B. *Related Works and Literature Survey*

Credit card fraud is a problem in the financial system. Credit card fraud costs a lot, gets more clever, and is very rare. Credit card fraud makes up half a percent of all transactions. Machine learning is used to identify fraud patterns in transaction data. Machine learning looks at transactions and tries to spot the ones that are fraud. The performance reported for machine learning is very different, across studies. The differences occur because the datasets are not the same, the way the researchers shape the data is not the same, the way the researchers select the dataset is not the same, and the way the researchers measure the results is not the same. The variability shows why a multi-model benchmarking framework is needed. The framework enforces pre-processing, sampling, and evaluation protocols. The framework ensure the comparisons are fair and reproducible. Models, including logistic regression and decision trees, have long supported fraud detection research. Classical supervised models are simple and easy to interpret. Classical supervised models often give good accuracy, low recall, for the minority fraud class [18], [19].

The paper on enhanced fraud detection using machine learning [22] authored by N. Doshi et al,. explores the use of ensemble learning methodologies to detect credit card fraud as a result of the worldwide increase in digital payments and consequent account theft. The paper introduces an ensemble system that integrates various machine learning algorithms and measures its performance against the baselines of individual models. The findings demonstrate that the proposed ensemble model, which does not use SMOTE, performs better than other methods, reaching an accuracy of 99.94%, thus being a strong candidate for imbalanced fraud detection scenarios.

In the survey paper [23] by N. Yousefi et al., goes over relevant methods in different sections. In the first part, the paper talks about traditional machine learning algorithms that use regular transaction features to predict fraud, which is usually based on some fixed attributes, such as user knowledge (knowledge-based methods) or accessible objects (object-based methods). The second part of the paper presents sophisticated user authentication methods using behavioral biometrics, which identify a person based on their unique way of interacting with electronic devices, which is inherently difficult to imitate.

The ensemble and mixed learning technique paper [24] by D.H.M. de Souza et al., solved the problem with a hybrid learning technique that employed K-means preprocessing for subsequent classification. Second, they presented a modified detector ensemble method that utilized OR-logic algorithm aggregation to increase the detection rate. The results revealed that the proposed methods reduced the computational cost and improved the performance compared with the latest techniques.

The work by T. R. Noviandy et al., on fraud detection using XGBoost drive ML [25] dives into the use of XGBoost for credit card fraud detection in the face of complex digital fraud and extreme class imbalance. A refurbished pipeline makes it possible to not only balance the data but also to augment it, with SMOTE - (Edited Nearest Neighbors) ENN intensifying the minority class signal and lessening the noise. The outcomes exhibit a high level of fraud detection, accompanied by good precision–recall trade-offs and decreased false negatives. This research is an application of XGBoost as a tool to develop stronger, faster, and fraud-resilient financial systems.

C. *Research Gaps*

After surveying the existing fraud detection literature, research gaps were identified. The first research gap is the lack

of a combined multi-model testing system. Most studies test models on datasets and metrics, and the different datasets and metrics make the results hard to compare. The second research gap is the use of real world datasets. Many studies rely on data or, data that are artificially balanced. The data limit how well the findings can be applied to different situations. Third, researchers have done limited comparisons of both modern algorithms under controlled conditions. This limited systematic comparison prevents understanding of how the model families differ when they are evaluated on inputs. Finally, few studies bring together imbalance-handling, feature scaling and, evaluation metrics into a pipeline that runs from start to finish. The community needs research to build evaluation frameworks that address imbalance-handling feature scaling and evaluation metrics and enable meaningful comparisons across studies.

III. Proposed Methodology

Several recent studies have been pointing to the necessity of robust data preparation, finding efficient methods for handling unbalanced data, and combining machine learning strategies to improve fraud detection capability in financial datasets that are highly skewed. In line with these recommendations, our approach involves meticulous data cleaning, feature standardization, generating more data via oversampling, employing multiple models for training, and Graphics Processing Unit (GPU) - accelerated hyperparameter tuning, all aimed at making the predictions as accurate as possible.

The entire flow is shown in Fig. 1. First, we cleaned the dataset by fixing missing values, removing duplicate records, and correcting unusual or incorrect entries. This step ensure that the model does not learn from messy or unreliable data. Next, we selected the most important features from the dataset so that the model focuses only on information that truly matters. Subsequently, we fed these selected features into our machine-learning model, which learns patterns and relationships within the data. This helps the model to understand how different factors affect the final result. Throughout the process, we used scaling, normalization, and other preprocessing steps to ensure that all the features were at the same level. These steps help the model learn better, avoid errors, and provide more stable predictions.

A. *Imbalance Handling Using SMOTE*

Fraud data are inherently imbalanced. To mitigate biased learning, we employed the Synthetic Minority Oversampling Technique (SMOTE) [3], which generates synthetic minority class samples within the scaled training set. This enhances the representation of fraudulent patterns without altering the original distribution. SMOTE is selectively applied only to models whose learning mechanisms are sensitive to class imbalance (e.g., LR, NB, MLP and KNN), whereas tree-based methods operate on the raw distribution.

B. *Multi-Model Development*

A collection of classical and ensemble classifiers was deployed to evaluate diverse learning patterns in fraud detection.

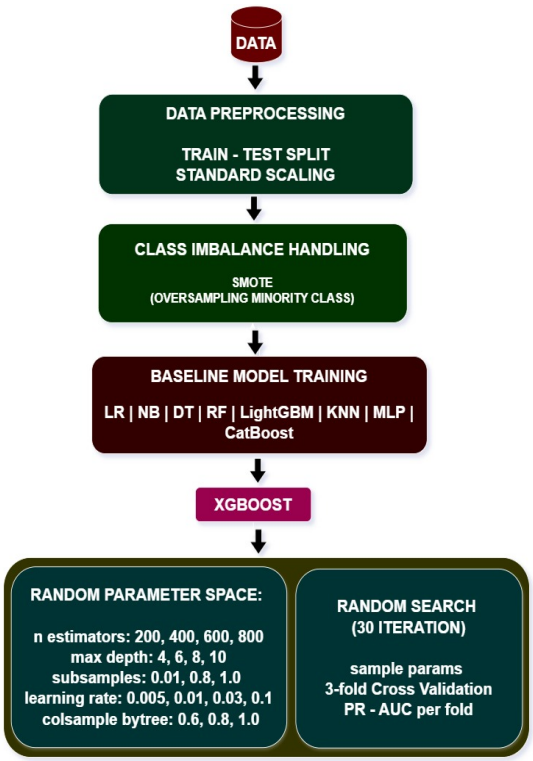

Fig. 1: Proposed Architecture

These include Logistic Regression, Naive Bayes, Decision Tree, Random Forest, KNN, MLP Neural Network, LightGBM and CatBoost. Each classifier receives the appropriate form of data (scaled or raw) and learns the fraud patterns independently. Model outputs, confusion matrices are stored to enable transparent comparisons across algorithms.

C. *Optimization*

A randomized hyperparameter space is defined, covering the boosting depth, learning rate, subsampling ratios, tree counts, and scale_pos_weight. A GPU-enabled manual randomized search was conducted for 30 iterations, each involving a three-fold cross-validation cycle optimized for PR-AUC. This approach captures non-linear relationships, manages class imbalances, and exploits parallel GPU computation to accelerate convergence. The best-performing configuration was retrained on the complete training dataset and archived as the final tuned model.

IV. Experiments

This section explains how the fraud-detection experiment was carried out from start to finish. It explains what data were used, how it was cleaned, how the best model settings were chosen, and which measurements were used to judge the models.

A. *Dataset*

An experiment-based design was used to evaluate the machine learning algorithms for credit card fraud detection.

The experiments were run with a very uneven credit card transaction dataset [25] that contained anonymous European cardholder records collected over two days. The credit card transaction dataset contained 284,807 transactions. Only 492 of these are fraud, which is about a 0.172 percent fraud rate as shown in Fig 2. The features are the 28 PCA-transformed components (V1–V28) plus the Time, the Amount, and binary class label.

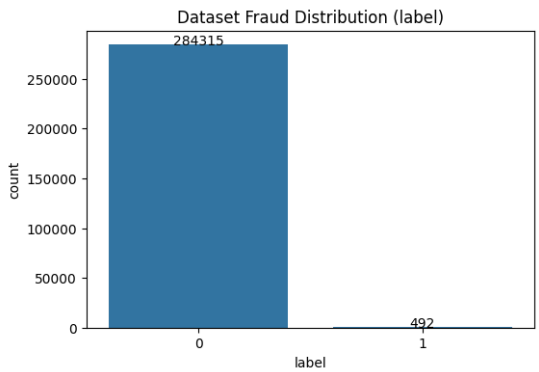

Fig. 2: Dataset Fraud Distribution

TABLE II: Descriptive Statistics of the Credit Card Fraud Dataset

Feature	Count	Mean	Std	Min	50% (Median)	Max
Time	284807	94813.86	47488.15	0.000000	84692.000000	172792.000000
V1	284807	0.000000001	1.958696	-56.407510	0.018109	2.454930
V2	284807	0.0000000003	1.651309	-72.715728	0.065486	22.057729
V3	284807	-0.000000001	1.516255	-48.325589	0.179846	9.382558
V4	284807	0.000000002	1.415869	-5.683171	-0.019847	16.875344
V5	284807	0.000000001	1.380247	-113.743307	-0.054336	34.801666
V6	284807	0.000000001	1.332271	-26.160506	-0.274187	73.301626
V7	284807	-0.000000001	1.237094	-43.557242	0.040103	120.589494
V8	284807	0.0000000001	1.194353	-73.216718	0.022358	20.007208
V9	284807	-0.000000002	1.098632	-13.434066	-0.051429	15.594995
V10	284807	0.000000002	1.088850	-24.588262	-0.092917	23.745136
V11	284807	0.000000002	1.020713	-4.797473	-0.032757	12.018913
V12	284807	-0.000000001	0.999201	-18.683715	0.140033	7.848392
V13	284807	0.0000000008	0.995274	-5.791881	-0.013568	7.126883
V14	284807	0.000000001	0.958596	-19.214325	0.050601	10.526766
V15	284807	0.000000005	0.915316	-4.498945	0.048072	8.877742
V16	284807	0.000000001	0.876253	-14.129855	0.066413	17.315112
V17	284807	-0.0000000004	0.849337	-25.162799	-0.065676	9.253526
V18	284807	0.000000001	0.838176	-9.498746	-0.003636	5.041069
V19	284807	0.000000001	0.814041	-7.213527	0.003735	5.591971
V20	284807	0.0000000006	0.770925	-54.497720	-0.062481	39.420904
V21	284807	0.0000000002	0.734524	-34.830382	-0.029450	27.202839
V22	284807	-0.0000000004	0.725702	-10.933144	0.006782	10.503090
V23	284807	0.0000000003	0.624460	-44.807735	-0.011193	22.528412
V24	284807	0.000000004	0.605647	-2.836627	0.040976	4.584549
V25	284807	0.0000000005	0.521278	-10.295397	0.016594	7.519589
V26	284807	0.000000002	0.482227	-2.604551	-0.052139	3.517346
V27	284807	-0.0000000004	0.403632	-22.565679	0.001342	31.612198
V28	284807	-0.0000000001	0.330083	-15.430084	0.011244	33.847808
Amount	284807	88.349620	250.120109	0.000000	22.000000	25691.160000
label	284807	0.001727	0.041527	0.000000	0.000000	1.000000

Table II presents the fundamental statistics for the dataset that records instances of credit card fraud. The columns represent features such as Time, Amount, and V1 through V28, and the figures show the behavior of the features in all 284,807 transactions. The ”Mean” gives the average value, and ”Std” shows how much the numbers are spread out. ”Min” and ”Max” indicate the smallest and the largest numbers locally. The middle value, called the ”Median,” indicates what is typical for a transaction. Most of the features (V1–V28) have average values close to zero because they were standardized by a technique that conceals sensitive information. The ”Amount” feature reveals that the majority of transactions are of low value, but a few are very high. Since the average value of the label is very low (0.0017), this implies that fraud cases are very few in the dataset.

TABLE III: Performance Metric Formulas

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \qquad \text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN} \qquad \text{F1} = 2 \times \frac{PR}{P + R}$$

$$\text{FPR} = \frac{FP}{FP + TN} \qquad \text{TPR} = \frac{TP}{TP + FN}$$

$$\text{ROC-AUC} = \int_0^1 TPR(FPR)\, d(FPR) \qquad \text{PR-AUC} = \int_0^1 P(R)\, d(R)$$

B. *Data Pre-Processing*

Data preparation began with the following steps. Stratified splitting was used to maintain class imbalance in the dataset. Variables were normalized so that they all had the same scale, which helped distance-based and gradient-based models. Because the class difference was large, resampling steps were added as needed. SMOTE [3] was applied to models that need extra minority data, which reduced sampling bias and kept noise low. Feature distributions were checked to ensure that original and the augmented samples looked the same. Additional steps were taken to locate missing values and ensure that there were no duplicates or damaged entries in the dataset. The scaling model was stored to ensure that everything remained the same during testing and when it was put into use. Each feature was examined to ensure that there was no data leakage between the training and testing datasets. The processed data were saved in two versions—original and scaled—so that models required the original numbers, such as tree-based models, could use them as required. The results of the preprocessing steps were finally documented and saved to facilitate the tracking of experiments and the reproducibility of the results across different runs.

C. *Hyper Parameter Details and Tuning*

The adjustment of the hyper parameter was done to find the best configuration that would allow the model to learn in the most efficient way. Initially, an 80:20 train–test split was used to divide the dataset. That is, 80% of the data were used to train the model and 20% were used for testing. The features were scaled prior to training using StandardScaler. GPU training is done with ‘tree_method = gpu_hist‘ and inference with ‘predictor = gpu_predictor‘ to achieve the maximum speed of operations. The learning objective is ‘binary : logistic‘ and the optimization metric is ‘logloss‘. Average Precision defined as ‘scoring = average_precision‘ is the main metric for model selection.

The tuned parameter grid evaluates ‘n_estimators‘ with values 200, 400, 600, and 800. Tree depth is limited by ‘max_depth‘ parameters with values 4, 6, 8, and 10. The

TABLE IV: Performance Comparison of Machine Learning Models

ID	Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
1	Naive Bayes	0.974404	0.056136	0.877551	0.105521	0.964582	0.087355
2	CatBoost	0.999596	0.92045	0.786122	0.842928	0.950232	0.838297
3	MLP Neural Network	0.999175	0.738318	0.806122	0.770732	0.964003	0.818736
4	LightGBM	0.978231	0.064787	0.867347	0.120567	0.906122	0.063562
5	Random Forest	0.999544	0.961538	0.765306	0.852273	0.956723	0.861023
6	KNN	0.997841	0.436548	0.877551	0.583051	0.948415	0.584669
7	Decision Tree	0.998929	0.676190	0.724490	0.699507	0.861946	0.490367
8	Logistic Regression	0.974106	0.057803	0.918367	0.108761	0.970843	0.724469
9	**XGBoost**	**0.999579**	**0.940476**	**0.806122**	**0.868132**	**0.975092**	**0.877494**

learning speed is controlled by ‘learning_rate‘ with values 0.005, 0.01, 0.03, and 0.1. Stability sampling tests ‘subsample‘ and ‘colsample_bytree‘ with values 0.6, 0.8, and 1.0. Split discipline is changed to ‘gamma‘ with values 0, 1, and 5, while child node control is ‘min_child_weight‘ with values 1, 3, and 5. The search picks 30 random parameter combinations (‘n_iter=30‘), and 3-fold cross-validation is used (‘cv=3‘). The best model along with the best parameters can be found in ‘best_estimator_‘ and ‘best_params_‘. SMOTE is deliberately not used during the tuning phase to enable a fair baseline comparison.

D. *Performance Metrics*

The performance evaluation followed standards for classification. The metrics as shown in Table III were accuracy, precision, recall, F1 score, ROC AUC, and PR AUC.

Confusion matrices, as shown in table V, were computed to see the prediction distribution. Threshold behavior was examined with ROC curves and Precision–Recall curves. The ROC curves and Precision–Recall curves showed how the model stayed stable as the decision boundaries changed. Fraud detection needed the insight because recall and precision drive the costs. Recall and precision matter a lot for fraud detection because missing fraud or raising alarms costs the business.

E. *Results and Discussion*

The Table IV compares nine machine-learning models in terms of accuracy, precision, recall, F1-score, ROC-AUC, and PR-AUC. The Fig. 3 represent how effectively each model is at detecting fraud. Higher numbers indicate better performance. The models may be clustered according to the similarity of their results across the metrics of evaluation. The group of best-performing models comprises XGBoost, Random Forest, and CatBoost since all the three models achieve very high levels of accuracy above 0.999 and also demonstrate their strong performances in precision, recall, and F1 scores. However, in this group, XGBoost is the one to lead, delivering the best harmony of precision, recall, F1, ROC-AUC, and PR-AUC. The second group consists of the MLP Neural Network and KNN - which are decent performers. Their recall values are high, but their precision and F1 scores are not as high as those of the top group. The third group comprises Decision Tree and Logistic Regression which have lower precision and weaker PR-AUC scores. Logistic Regression have high recall, but its precision is very low.

TABLE V: Confusion Matrix Values

Model	**TP**	**FP**	**FN**	**TN**
Naive Bayes	86	1446	12	55418
CatBoost	81	7	22	56858
MLP Neural Network	79	28	19	56836
LightGBM	85	1227	13	55637
Random Forest	75	3	23	56861
KNN	86	111	12	56753
Decision Tree	71	34	27	56830
Logistic Regression	90	1467	8	55397
XGBoost	79	5	19	56859

XGBoost is the model that performs best in this research as it consistently yields the best results across all the main performance metrics. It provides very high accuracy and retrieves the majority of real fraud cases without omitting them. Another reason for which XGBoost is so outstanding is that it obtains high ROC-AUC and PR-AUC scores. XGBoost can effectively differentiate between fraudulent and non-fraudulent cases even when there are very few fraudulent values in the data. .

V. Conclusion

A multi model benchmarking framework for credit card fraud detection is introduced. Ensemble-based tree models — XGBoost, CatBoost and Random Forest were the reliable and the most consistent under class imbalance. PR-AUC and recall should be prioritized over accuracy in domains. Although limited by a single dataset and fixed hyper parameters, the

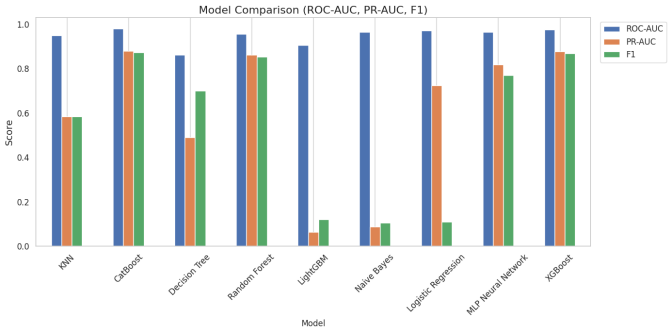

Fig. 3: Model Comparison – Bar Chart

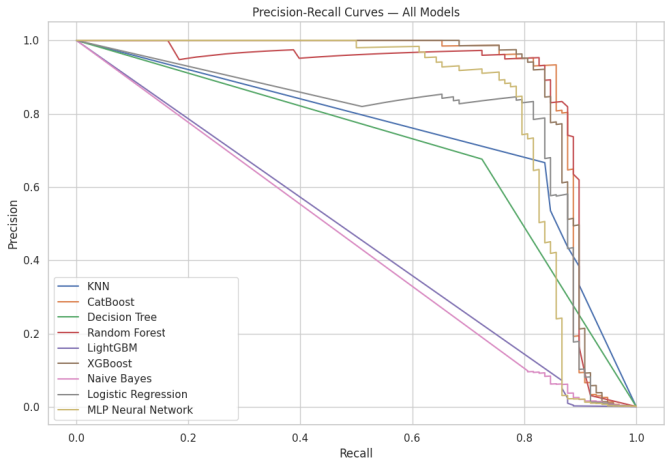

Fig. 4: Precision–Recall Curve

framework provides a strong foundation for future research on adaptive, interpretable, and scalable fraud detection systems.

Building on this work in the future could involve adding different models that can handle sequences over time, such as Transformers or LSTMs, in order to better understand and represent the time-based patterns in transactions. A graph-based approach utilizing the relationships between customers, merchants, and devices can be very helpful in uncovering the most concealed connections that are related to fraudulent activities.

References

[1] E. Strelcenia and S. Prakoonwit, "Improving Classification Performance in Credit Card Fraud Detection by Using New Data Augmentation," AI, vol. 4, no. 1, pp. 172–198, 2023. doi: 10.3390/ai4010008.

[2] X. Niu, L. Wang, and X. Yang, "A Comparison Study of Credit Card Fraud Detection: Supervised versus Unsupervised," arXiv:1904.10604, 2019.

[3] M. Zhu, Y. Zhang, Y. Gong, C. Xu, and Y. Xiang, "Enhancing Credit Card Fraud Detection: A Neural Network and SMOTE Integrated Approach," arXiv:2405.00026, 2024.

[4] V. S˙INAP, "Comparative analysis of machine learning techniques for credit card fraud detection: Dealing with imbalanced datasets," Turkish Journal of Engineering, vol. 8, no. 2, pp. 196–208, 2024.

[5] S. F. Farabi et al., "Enhancing Credit Card Fraud Detection: A Comprehensive Study of Machine Learning Algorithms and Performance Evaluation," Journal of Business and Management Studies, vol. 6, no. 3, pp. 252–259, 2024.

[6] A. Sani, Z. L. Hassan, and A. T. Balarabe, "A Logistic Regression-based Model for Identifying Credit Card Fraudulent Transactions," Asian Journal of Research in Computer Science, vol. 17, no. 7, pp. 41–54, 2024.

[7] S. S. Dhatrak, M. Patil, M. Bodke, S. A. Pathan, and S. A. Pathan, "Credit Card Fraud Detection Using State-of-the-Art Machine Learning and Deep Learning Algorithms," IJARSCT, pp. 229–233, 2024.

[8] I. D. Mienye and Y. Sun, "A Deep Learning Ensemble With Data Resampling for Credit Card Fraud Detection," IEEE Access, vol. 11, pp. 30628–30638, 2023.

[9] S. Verma and J. Dhar, "Credit Card Fraud Detection: A Deep Learning Approach," arXiv:2409.13406, 2024.

[10] M. Habibpour et al., "Uncertainty-Aware Credit Card Fraud Detection Using Deep Learning," arXiv:2107.13508, 2021.

[11] S. F. Ahmed et al., "Unveiling the Frontiers of Deep Learning: Innovations Shaping Diverse Domains," arXiv:2309.02712, 2023.

[12] I. D. Mienye and T. G. Swart, "A Hybrid Deep Learning Approach with GAN for Credit Card Fraud Detection," Technologies, vol. 12, no. 10, p. 186, 2024.

[13] M. Jayalakshmi, R. Kumar, and T. Thanushree, "A Survey on Credit Card Fraud Detection using Deep Learning Model," Deleted Journal, vol. 3, no. 4, pp. 1325–1334, 2025.

[14] T. T. Nguyen, H. Tahir, M. Abdelrazek, and A. Babar, "Deep Learning Methods for Credit Card Fraud Detection," arXiv:2012.03754, 2020.

[15] S. Banka et al., "Securing Fintech: A Machine Learning Approach for Credit Card Fraud Detection," IEEE ICC, pp. 814–821, 2024.

[16] "Credit Card Fraud Detection using Machine Learning," IJARSCT, pp. 142–147, 2023.

[17] "Credit Card Fraudulent Transaction Detection and Prevention," IJSTE, vol. 11, no. 4, pp. 3255–3260, 2023.

[18] N. Saranya, M. Devi, A. Mythili, and S. H., "Data science and machine learning methods for detecting credit card fraud," The Scientific Temper, vol. 14, no. 3, pp. 840–844, 2023.

[19] S. Fathima, L. C. Sekhar, and J. K. U, "Credit Card Fraud Detection – A Machine Learning Perspective," IJSR, 2023.

[20] Sk. Asiff, "Credit Card Fraud Detection using Ensemble Machine Learning Method – Gradient Boosting Framework," IJSTE, vol. 11, no. 6, pp. 4807–4810, 2023.

[21] F. Aslam, "Advancing Credit Card Fraud Detection: A Review of Machine Learning Algorithms and the Power of LightGBM," AJCS&T, 2024.

[22] N. Doshi, G. Chillarge, and S. Shekapure, "Enhanced Detection of Credit Card Fraud Using Machine Learning Techniques," IJSTE, vol. 12, no. 5, pp. 5542–5545, 2024.

[23] N. Yousefi, M. Alaghband, and I. Garibay, "A Comprehensive Survey on ML Techniques for Credit Card Fraud Detection," arXiv:1912.02629, 2019.

[24] D. H. M. de Souza and C. J. Bordin, "Ensemble and Mixed Learning Techniques for Credit Card Fraud Detection," arXiv:2112.02627, 2021.

[25] T. R. Noviandy et al., "Credit Card Fraud Detection Using XGBoost-Driven ML and Data Augmentation Techniques," 2023

[26] M. K. H. Chy and O. Buadi, "A Machine Learning Driven Website Platform and Browser Extension for Real-time Fraud Detection," arXiv:2411.00368, 2024.

[27] A. R. Khalid et al., "Enhancing Credit Card Fraud Detection: An Ensemble Machine Learning Approach," Big Data and Cognitive Computing, 2024.

[28] K. G. Dastidar, O. Caelen, and M. Granitzer, "Machine Learning Methods for Credit Card Fraud Detection: A Survey," IEEE Access, 2024.